

Global Variables

(Lecture 14)

```
#include <iostream>
using namespace std;
```

```
int x = 100;
```

```
void fun() {
    cout << x << endl;
}
```

```
int main() {
    cout << x << endl;
    fun();
}
```

Summary Notes

By - Raghav Sir

Global Variables

```
#include<iostream>
using namespace std;
int x = 10; // Global Variable
int main(){
    ✓ cout<<x<<endl;
    ✓ int x = 80;
    ✓ cout<<x<<endl;
}
```

10
x

80
x

Output

10

80

Nested Scope (Variable Shadowing)



Earn ₹s

```
int main() {
```

```
    int x = 10;
```

```
{
```

```
    int x = 20;
```

```
    cout << x << endl;
```

```
}
```

```
    cout << x << endl;
```

```
}
```

Accessing a shadowed Global Variable

```
#include <iostream>
```

```
using namespace std;
```

```
int x = 50;
```

```
int main() {
```

```
    int x = 10;
```

```
    cout << x << endl;
```

```
    cout << ::x << endl;
```

```
}
```

Multiple functions having **same** variable name

```
void fun() {  
    int x = 5;  
    cout << x << endl;  
}
```

```
void display() {  
    int x = 10;  
    cout << x << endl;  
}
```

```
int main() {  
    fun();  
    display();  
}
```

Default Parameters

```
#include <iostream>
using namespace std;

void power(int base, int exponent = 2) {
    int ans = 1;
    for (int i = 0; i < exponent; i++)
        ans *= base;
    cout << ans << endl;
}
```

```
int main() {
    power(5);
    power(5, 3);
}
```

→ *power(5, 2)*

Mark True or False

1. The variables commonly used in C++ functions are available to all the functions in a program. *False*
2. To return the control back to the calling function we must use the keyword return. *False*
3. The same variable names can be used in different functions without any conflict. *True*
4. Every called function must contain a return statement. *False*
5. A function may contain more than one return statements. *True*
6. Each return statement in a function may return a different value. *True*

Function Overloading (Case-1)

```
void print() {  
    cout << "No arguments" << endl;  
}
```

```
void print(int x) {  
    cout << "One integer: " << x << endl;  
}
```

```
void print(int x, int y) {  
    cout << "Two integers: " << x << " " << y << endl;  
}
```


Function Overloading (Case-2)

```
void display(int x) {  
    cout << "Integer: " << x << endl;  
}
```

```
void display(double x) {  
    cout << "Double: " << x << endl;  
}
```

```
void display(char x) {  
    cout << "Character: " << x << endl;  
}
```

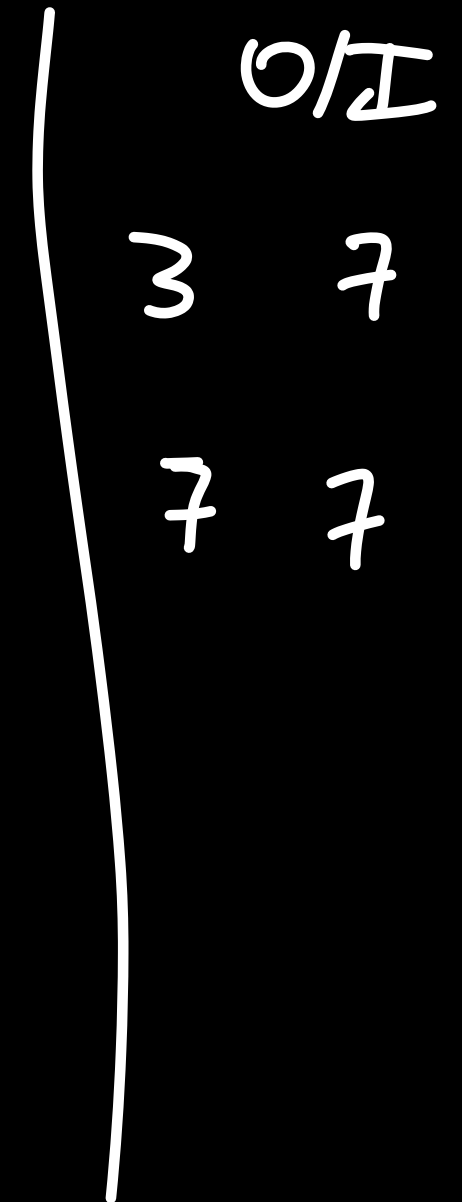
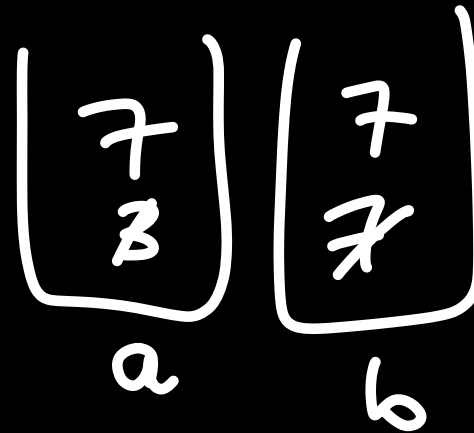
Function Overloading (Case-3)

```
void fun(int x, char ch) {  
    cout << "int, char" << endl;  
}
```

```
void fun(char ch, int x) {  
    cout << "char, int" << endl;  
}
```

Ques: Swap 2 Numbers

```
✓ int a, b;  
✓ cin >> a >> b;  
✓ a = b;  
✓ b = a;  
✓ cout << a << " " << b;
```





Ques: Swap 2 Numbers

```
✓ int a, b;  
✓ cin >> a >> b;  
✓ a = a + b;  
✓ b = a - b;  
✓ a = a - b;  
cout << a << " " << b;
```

$$\begin{array}{|l} 2+5-2 \\ 2+5 \\ 2 \end{array} \quad \begin{array}{|l} 2+8-8 \\ 8 \end{array}$$

$a \qquad b$

output

2 5

$$a = (a+b) - (b = a)$$

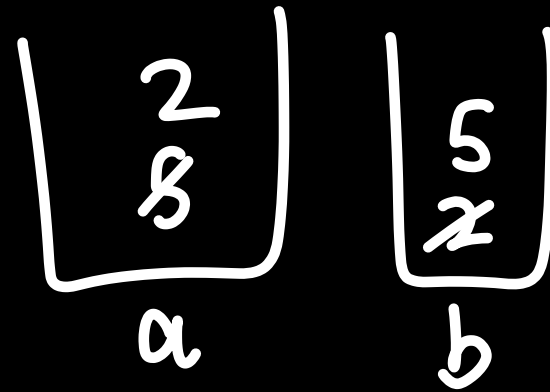
one liner

Ques: Swap 2 Numbers

✓ `int a = 5;`

✓ `int b = 2;`

✓ `a = (a + b) - (b = a)`
7 5

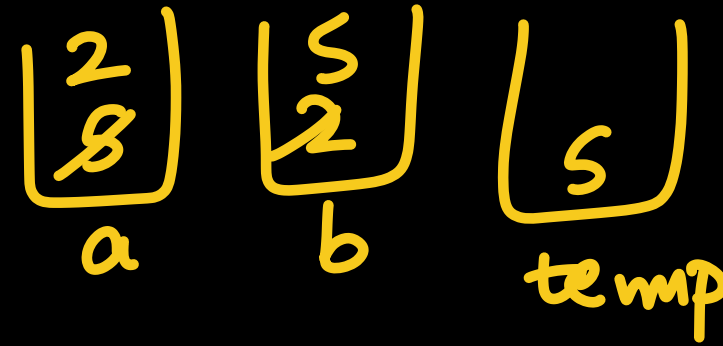


Why does this **not** work ?

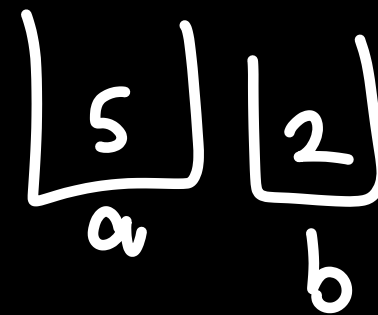
```

void swap(int5 a, int2 b){
    ✓ int temp = a;
    ✓ a = b;
    ✓ b = temp;
}

int main(){
    ✓ int a = 5, b = 2;
    ✓ swap(5, 2);
    ✓ cout<<a<<" "<<b<<endl;
}
    
```



Output
5, 2



Is there a way to solve this ?

What if we are able to store or pass the actual address inside functions?